

Senior DevOps / SRE Interview Questions & Answers — Part 6

Advanced Staff/Lead-level production engineering interview preparation.

161. How do you debug high memory usage on Linux servers?

Answer

I first identify whether memory usage is:

- application memory
- page cache
- kernel memory
- memory leak

Commands:

```
free -m  
vmstat  
ps aux --sort=-%mem
```

Then I investigate:

- heap growth
- cache pressure
- swap activity
- OOM events

High memory usage alone is not always a problem.

The key is whether the system is under pressure.

162. Explain load average in Linux.

Answer

Load average represents:

- running processes
- waiting processes

Not just CPU utilization.

High load with low CPU often indicates:

- IO wait
- lock contention
- blocked processes

I correlate:

- CPU
- IO
- process states

before concluding root cause.

163. How do you troubleshoot high IO wait?

Answer

High IO wait indicates processes blocked on storage.

Checks:

```
iostat -xz 1  
iotop
```

Common causes:

- slow disks
- excessive logging
- database contention
- backups
- network storage latency

Mitigation:

- optimize queries
 - improve storage
 - cache hot data
 - isolate workloads
-

164. Explain Linux cgroups.

Answer

cgroups control resource allocation.

Used for:

- CPU limits
- memory limits
- IO throttling

Containers rely heavily on cgroups.

Understanding cgroups helps debug:

- throttling
 - OOM issues
 - noisy neighbors
-

165. How do you debug file descriptor exhaustion?

Answer

Symptoms:

- "too many open files"
- connection failures
- app instability

Commands:

```
lsof  
ulimit -n
```

Common causes:

- connection leaks
- unclosed files
- traffic spikes

Mitigation:

- increase limits carefully
- fix leaks
- optimize connection handling

166. Explain swap usage and why it can be dangerous.

Answer

Swap extends memory using disk.

Heavy swap usage causes:

- latency spikes
- system slowdown
- thrashing

In Kubernetes production nodes, I usually avoid swap.

Excessive swap often hides memory pressure temporarily while degrading performance.

167. How do you troubleshoot network packet drops?

Answer

I investigate:

- NIC errors
- MTU mismatch
- congestion
- firewall rules
- kernel buffers

Commands:

```
netstat -s  
sar -n DEV  
ethtool
```

Packet loss often causes intermittent latency and retry storms.

168. Explain TCP handshake and debugging connection failures.

Answer

TCP handshake:

1. SYN
2. SYN-ACK
3. ACK

Failures may occur due to:

- firewall blocks
- SYN backlog exhaustion
- packet drops
- DNS failures

Tools:

- tcpdump
- netstat
- ss

Networking issues require packet-level thinking.

169. How do you debug SSL/TLS handshake failures?

Answer

I validate:

- certificate validity
- trust chain
- protocol mismatch
- cipher compatibility
- expiration

Commands:

```
openssl s_client
```

TLS failures are common during certificate rotation.

170. Explain DNS caching issues.

Answer

DNS caching can cause:

- stale records
- delayed failover
- inconsistent routing

I validate:

- TTL settings
- local resolver cache

- CoreDNS cache
- upstream propagation

DNS is often underestimated during outages.

171. How do you troubleshoot Kafka broker failures?

Answer

I check:

- broker health
- replication status
- ISR shrinkage
- disk usage
- network latency

Common issues:

- disk saturation
- GC pauses
- controller instability

Kafka failures can cascade across dependent systems quickly.

172. Explain Kafka partitioning strategy.

Answer

Partitioning affects:

- scalability
- ordering
- throughput

Key design considerations:

- balanced partitions
- consumer parallelism
- key distribution

Bad partitioning creates hotspots.

173. How do you handle replay storms in event-driven systems?

Answer

Replay storms can overload downstream systems.

Mitigations:

- rate limiting
- backpressure
- replay throttling
- queue buffering

Event systems require careful failure handling.

174. Explain backpressure in distributed systems.

Answer

Backpressure prevents producers from overwhelming consumers.

Without backpressure:

- queues explode
- latency spikes
- cascading failures occur

Techniques:

- bounded queues
 - rate limiting
 - adaptive throttling
-

175. How do you debug distributed transaction failures?

Answer

I trace:

- request flow
- retries
- message ordering
- idempotency handling

Distributed systems require eventual consistency thinking.

I avoid tightly coupled synchronous dependencies wherever possible.

176. Explain CAP theorem in practical terms.

Answer

Distributed systems can prioritize:

- consistency
- availability
- partition tolerance

During partitions:

- choose consistency or availability

Real-world systems make tradeoffs based on business requirements.

177. How do you design idempotent systems?

Answer

Idempotency ensures repeated requests produce same outcome.

Techniques:

- request IDs

- deduplication
- transaction tracking

Critical for:

- retries
 - event systems
 - payment systems
-

178. Explain eventual consistency.

Answer

Eventual consistency means replicas converge over time.

Advantages:

- scalability
- availability

Challenges:

- stale reads
- conflict resolution

System design must account for temporary inconsistency.

179. How do you troubleshoot distributed cache inconsistency?

Answer

Checks:

- cache invalidation
- replication lag
- TTL mismatch
- stale writes

Cache consistency bugs are often subtle and intermittent.

180. Explain leader election in distributed systems.

Answer

Leader election coordinates distributed control.

Used in:

- Kubernetes controllers
- Kafka
- databases

Failures during election can cause instability or split brain.

181. What is split-brain scenario?

Answer

Split brain happens when multiple nodes think they are leader.

Risks:

- data corruption
- inconsistent writes

Prevented using:

- quorum
 - fencing
 - consensus protocols
-

182. Explain quorum in distributed systems.

Answer

Quorum ensures majority agreement.

Examples:

- etcd
- Kafka ISR

- distributed databases

Quorum improves consistency and failure safety.

183. How do you handle partial outages?

Answer

Partial outages are harder than full outages.

My approach:

1. identify affected segments
2. isolate blast radius
3. shift traffic
4. degrade gracefully
5. maintain communication

Observability becomes critical during partial failures.

184. Explain graceful shutdown handling.

Answer

Applications should:

- stop accepting traffic
- finish active requests
- close connections safely

Improper shutdown handling causes:

- dropped requests
- data corruption

Critical for rolling deployments.

185. How do you troubleshoot zombie processes?

Answer

Zombie processes are exited processes awaiting cleanup.

Commands:

```
ps -ef
```

Usually caused by:

- parent process bugs
- improper signal handling

Large zombie buildup may indicate application defects.

186. Explain kernel panic and recovery strategy.

Answer

Kernel panic indicates critical kernel failure.

Possible causes:

- hardware issues
- driver bugs
- kernel corruption

Recovery:

- reboot safely
- collect crash dumps
- analyze logs
- isolate faulty hardware/software

Kernel panics require deep systems investigation.

187. How do you debug intermittent network latency?

Answer

I correlate:

- packet loss
- retransmissions
- congestion
- DNS latency
- routing instability

Tools:

- traceroute
- tcpdump
- ping
- mtr

Intermittent issues require historical analysis.

188. Explain SYN flood attacks and mitigation.

Answer

SYN flood overwhelms connection queues.

Mitigation:

- SYN cookies
- rate limiting
- WAF
- load balancing
- DDoS protection

Security and reliability overlap heavily here.

189. How do you secure Linux servers?

Answer

Controls:

- least privilege
- SSH hardening
- patching
- MFA
- audit logging
- firewalling
- intrusion detection

Security is continuous operational discipline.

190. Explain bastion hosts and alternatives.

Answer

Bastions provide controlled administrative access.

Modern alternatives:

- AWS SSM
- Zero Trust access
- VPNless access

I prefer eliminating direct SSH exposure whenever possible.

191. How do you handle incident communication during outages?

Answer

Good communication reduces chaos.

Principles:

- clear ownership

- regular updates
- factual status
- no speculation

I separate:

- technical debugging
- stakeholder communication

Strong communication is critical for leadership roles.

192. Explain blameless postmortems.

Answer

Focus:

- system failures
- process gaps
- detection weaknesses

Not individual blame.

Blameless culture improves reliability learning.

193. How do you prioritize reliability work vs feature delivery?

Answer

I use:

- SLOs
- error budgets
- business impact
- operational risk

Reliability investment should align with business priorities.

194. Explain toil in SRE.

Answer

Toil is repetitive manual operational work.

Examples:

- repetitive deployments
- manual recovery
- repetitive ticket handling

Goal:

Automate toil to improve engineering efficiency.

195. How do you measure platform engineering success?

Answer

Metrics:

- deployment frequency
- MTTR
- lead time
- developer satisfaction
- operational load reduction

Platform engineering exists to improve developer productivity safely.

196. Explain DORA metrics.

Answer

Key DevOps metrics:

1. deployment frequency
2. lead time
3. MTTR

4. change failure rate

These measure engineering delivery performance.

197. How do you improve deployment frequency safely?

Answer

Strategies:

- automation
- feature flags
- canary releases
- strong testing
- observability

Fast delivery without reliability is dangerous.

198. How do you handle burnout in on-call teams?

Answer

I reduce:

- alert fatigue
- repetitive toil
- unclear ownership

Practices:

- balanced rotations
- better automation
- incident reviews
- sustainable paging policies

Healthy operational culture matters.

199. What makes a strong RCA document?

Answer

A strong RCA includes:

- timeline
- trigger
- contributing factors
- detection gaps
- recovery actions
- prevention items

Good RCAs improve systems permanently.

200. What separates elite SRE/Platform teams from average ones?

Answer

Elite teams focus on:

- automation
- observability
- operational discipline
- resilience engineering
- platform enablement
- reliability culture

Most importantly:

They continuously reduce operational complexity while improving developer velocity and system reliability.